

CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

Permissions And Blast Radius

Claude Code deep dive, episode 4

You can undo what changed a file. You cannot undo what left the machine.

You have an undo, you have pressed it, and it worked. That experience quietly teaches you that mistakes here are cheap. Most of them are.

There is a category this tool can reach that no undo covers, and by the time you notice it has already happened. This sheet is that boundary, drawn precisely.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- What checkpoints actually cover, and the narrower-than-expected edge
- Why checkpoints and git are two independent histories
- The things outside the filesystem entirely
- The four permission modes
- Which mode you are probably on right now
- What plan mode is actually for
- Allow-listing, and settings scope

Out of scope

- Organisation policy and audit, which is episode 15
- Sandboxing configuration in detail
- MCP server trust, which is episode 7
- Any claim that a permission mode makes an action safe

The question before any task is not whether you trust the agent. It is what this particular task can reach.

01 The four modes

Auto	a classifier blocks the risky ones
Manual	asks before edits and shell commands
Accept edits	edits and common filesystem commands, no prompt
Plan	explores and proposes, edits nothing
Cycle them with	Shift+Tab
Default on Pro, Max, Team	Auto, for terminal and VS Code

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.



03 The good news, and it is genuinely good

Before it edits a file, it snapshots what was there. Press Escape twice to rewind, or ask it to undo.

That is not a git operation and it does not require you to have committed, which matters because the moment you most want an undo is the moment you were halfway through something and had not committed for an hour.

They survive resuming a conversation, so yesterday's snapshots are still there this morning.

Before Claude edits a file it snapshots the current contents; pressing Esc twice rewinds to a previous state.

Claude Code docs, How Claude Code works, checked September 2026

Checkpoints are separate from git and remain available when you resume a conversation.

Claude Code docs, How Claude Code works, checked September 2026

04 The edge is narrower than "file changes"

This is the correction worth the whole sheet. Checkpoints track changes made through CLAUDE'S OWN FILE EDITING TOOLS. Changes made through Bash commands, or by any other process, are not captured.

So a fix applied with a stream editor in a shell command is a file change that is NOT in the snapshot.

The documentation is blunt about what follows: this is not a replacement for git. There is one further gap: a restore skips symlinked and hard linked files.

Checkpoints only track changes made through Claude's file editing tools. Changes made through Bash commands or external processes are not captured. This isn't a replacement for git.

Claude Code docs, Best practices, checked September 2026

A restore skips symlinked and hard-linked files.

Claude Code docs, How Claude Code works, checked September 2026

05 And everything here is outside it

A request sent to an API. A row written to a production database. A deployment triggered. A message posted. An email sent. A package published. A cloud resource created, and billed for.

What they have in common is not that they are dangerous. It is that they are outside your filesystem, and a snapshot of your files cannot reach any of them.

You control those with your permission mode and your permission rules instead.

Actions that affect remote systems, such as databases, APIs and deployments, cannot be checkpointed; you control those with your permission mode and permission rules.

`Claude Code docs, How Claude Code works, checked September 2026`

06 The mode you are probably on

At the time of writing, Auto is the built-in starting mode on Pro, Max and Team plans for interactive terminal and VS Code sessions.

A classifier reviews most actions in the background and blocks the risky ones rather than stopping to ask you about each.

That is a reasonable default, and it is worth understanding what it means: you are not being asked, by design. Fewer prompts is not evidence that nothing consequential is happening.

In Auto mode a classifier reviews most actions in the background and blocks the risky ones instead of asking you.

`Claude Code docs, Permission modes, checked September 2026`

On Pro, Max and Team plans, Auto is the built-in starting permission mode for interactive terminal and VS Code sessions.

`Claude Code docs, Permission modes, checked September 2026`



07 Plan mode is not training wheels

In plan mode it explores and proposes without editing your source files.

The instinct is to treat that as something you graduate from once you trust the tool. It is closer to the opposite.

Think about when not editing is exactly what you want: a codebase you do not know yet, a change whose shape you have not decided, an investigation where you want the analysis before any commitment. It is the mode for work where thinking is the deliverable.

In plan mode Claude explores and proposes a plan without editing your source files.

Claude Code docs, Permission modes, checked September 2026

Use plan mode to separate exploration from execution, because letting Claude jump straight to coding can produce code that solves the wrong problem.

Claude Code docs, Best practices, checked September 2026

08 So the prompts you do see are worth reading

Allow specific commands you trust in settings, so it stops asking about them. Running your test suite. Checking git status.

A prompt you always approve is a prompt you have stopped reading, and a prompt you have stopped reading provides no safety at all.

Allow-listing the boring ones is not laziness. It keeps the remaining prompts rare enough that you still look at them.

You can allow specific commands in settings so Claude does not ask each time, which is useful for trusted commands like npm test or git status.

Claude Code docs, How Claude Code works, checked September 2026

A prompt that is always approved provides no safety, because approval without reading is indistinguishable from no prompt at all.

First principles, not vendor documentation



09 And the moment this is more than you

Settings are scoped, from organisation-wide policy down to personal preference.

So a team can say at the top level that certain things are never permitted here, and that is not a convention in a README that everyone agrees to and then forgets. It is enforced configuration.

If you are introducing this to a team, that scope is the conversation to have first, before anybody has opinions about prompts.

Settings can be scoped from organization-wide policies down to personal preferences.

Claude Code docs, How Claude Code works, checked September 2026

Before you start: what can this task reach?

1. Does anything in this task leave the filesystem?
2. Will any file be changed via Bash rather than the edit tools?
3. Have you committed recently enough that git is a real fallback?
4. Which permission mode are you actually in?
5. If it is Auto, are you comfortable not being asked?
6. Should this start in plan mode instead?
7. Are the boring commands allow-listed so the real prompts stand out?
8. For a team: is the policy set at organisation level, or per person?

Everything else

- | | |
|--|---|
|  Site
https://code-with-sam-dev.github.io |  Instagram
https://www.instagram.com/codewithsamdev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  Facebook
https://www.facebook.com/codewithsamdev |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  GitHub
https://github.com/code-with-sam-dev |
|  x
https://x.com/CodeWithSamDev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  TikTok
https://www.tiktok.com/@codewithsamdev |  Buy me sadza
https://buymesadza.com/codewithsam |
| | Permission modes
https://code.claude.com/docs/en/permission-modes |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.